C++: Siklus Hidup Objek

IF2210 – Semester II 2025/2026

Sumber: Diktat Bahasa C++ oleh Hans Dulimarta

Modernisasi oleh Satrio Adi Rukmono

Siklus Hidup Objek

Siklus Hidup Objek

- Sesi lalu kita membahas:
  - objek sebagai entitas dengan tanggung jawab,
  - *class* sebagai keputusan desain.
- Sesi ini kita bertanya:
  - Apa yang terjadi saat objek lahir, disalin, dipindahkan, dan mati?
- *Lifecycle* adalah bagian dari desain.

Constructor: Menjamin *Invariant* Saat Lahir

- *Constructor* (di C++ umum disebut “ctor”) adalah mekanisme untuk memastikan bahwa objek lahir dalam keadaan valid.
- Kembali ke contoh *stack*.
  - Apa *invariant* sebuah *stack*?
    - Tidak boleh memiliki elemen di bawah indeks 0.
    - Struktur internal harus konsisten.
    - Data harus siap menerima operasi.
  - *Constructor* bertanggung jawab menjamin semua itu benar saat objek dibuat.
- *Constructor* bukan sekadar mengisi nilai, tapi menjamin bahwa objek siap menjalankan tanggung jawabnya.

Destructor

- *Destructor* (di C++ umum disebut “dtor”) memastikan bahwa ketika objek mati, ia melepaskan semua tanggung jawab yang dimilikinya.

Contoh: Deklarasi ctor, dtor

- Untuk mengilustrasikan ctor, mari sejenak pakai raw array.

```
class Stack {
  public:
    Stack();     // deklarasi default ctor
    Stack(int); // deklarasi ctor dengan 1 parameter
    ~Stack();   // deklarasi dtor (wajib jika mengelola raw
resource)
    // ... methods
  private:
    int capacity_{0};
    int top_{0};
    int* data_; // raw array!
};
```

caveat: *in real code*, pakai `size_t` untuk mencegah index negatif:

```
#include <cstddef>
...
std::size_t
capacity_{0};
std::size_t top_{0};
```

Contoh: Implementasi ctor, dtor

```
Stack::Stack() : Stack(16) {}      // definisi default ctor
                                   // - pemakaian:
                                   //     Stack s;

Stack::Stack(int capacity)         // definisi ctor 1 param
  : capacity_{capacity},           // - pemakaian:
    top_{0},                       //     Stack s{32}; // modern
C++
    data_{new int[capacity_]} {}   //     Stack s(32); // old C++

Stack::~Stack() {                  // definisi dtor
    delete[] data_;                // - otomatis dieksekusi saat
objek
}                                  //   keluar dari scope { }
```

Initialization List vs Assignment di Body Constructor

- Dua Cara Menginisialisasi Data Member

```
// Cara 1: Initialization list
// (direkomendasikan)

Stack::Stack(int capacity)
    : capacity_{capacity},
      top_{0},
      data_{new int[capacity_]}
{}
```

```
// Cara 2: Assignment di body

Stack::Stack(int capacity) {
    capacity_ = capacity;
    top_ = 0;
    data_ = new int[capacity_];
}
```

Apa Bedanya?

1. **Urutan Eksekusi**
   - Dalam C++:

     Data member selalu diinisialisasi sebelum body constructor dijalankan.
   - Artinya pada cara kedua:
     - `capacity_`, `top_`, `data_` terlebih dahulu **dibuat dengan default initialization**
     - lalu di-assign ulang di dalam body
   - Pada cara pertama:
     - Data member langsung dibangun dengan nilai yang diinginkan

Apa Bedanya?

2. **Efisiensi**
   - Untuk tipe sederhana (`int`), perbedaan kecil.
   - Untuk tipe kompleks (`std::vector`, `std::string`, dll.):
     - Initialization list:
       - langsung membangun objek dengan nilai final
     - Assignment di body:
       - bangun default dulu
       - lalu assign ulang

       Bisa lebih mahal.

Apa Bedanya?

3. **Wajib untuk const dan reference**
   - Ini tidak bisa dilakukan:

```
const int x;

Stack(int c) {
    x = c;   // ERROR
}
```

   - Harus:

```
Stack(int c) : x{c} {}
```

Apa Bedanya?

4. **Hubungan dengan Invariant**
   - Constructor bertugas:

     Menjamin object invariant sejak awal.
   - Initialization list membantu memastikan:
     - Objek lahir dalam keadaan lengkap.
     - Tidak ada fase “setengah jadi”.
   - Secara desain:

     Initialization list lebih konsisten dengan gagasan bahwa objek lahir sudah valid.

Siklus Hidup Objek

- RAII: *Resource Acquisition Is Initialization*
- Dalam C++, siklus hidup *resource* terikat pada siklus hidup objek
  - ctor → mengakuisisi *resource* dan menjamin *invariant*
  - dtor → melepaskan *resource*
  - *scope* menentukan panjang usia objek
- Tanggung jawab atas *resource* ada pada objek yang memilikinya
- Jika menggunakan *resource* mentah (mis. *raw array*), implementasi kelas dibebani pengelolaan *lifecycle* tersebut

Rule of 3 / 5 / 0

- **Rule of 3 (C++ klasik)**
  - Jika perlu mendefinisikan dtor, cctor, atau copy assignment (operator=),
    biasanya perlu mendefinisikan ketiganya.
  - Ini terjadi karena kelas mengelola *resource* mentah.
- **Rule of 5 (sejak C++11)**
  - Dengan *move semantics*, ditambahkan *move constructor* & *move assignment*
  - Memungkinkan pemindahan kepemilikan *resource* tanpa penyalinan mahal.

Rule of 3 / 5 / 0

- **Rule of 0 (C++ modern)**
  - Hindari *resource* mentah.
  - Gunakan tipe RAII (`std::vector`, `std::string`, `std::unique_ptr`, dll.)
  - Tidak perlu mengimplementasikan dtor, cctor, dst. secara manual.
  - *Lifecycle* data member ditangani oleh kelasnya masing-masing.
- Jika Anda menulis `new` di dalam kelas, hampir pasti Anda harus menulis destructor, copy, dan move juga.
  - Kompleksitas *special member functions* adalah konsekuensi dari keputusan desain mengelola *resource* mentah.

Copy Constructor

- *Copy constructor* (“cctor”) membuat objek baru dengan identitas baru, tapi *state* yang sama (duplikat) dari objek yang sudah ada.
- Kapan dieksekusi?
  - Deklarasi variabel dengan inisialisasi,
    - e.g. `Stack s2 = s1;`
  - *Passing parameter* aktual ke parameter formal secara *pass by value*
  - Pemberian *return value* dari *method* yang bukan return ptr / ref
- Pada C++ modern, dalam banyak kasus *copy* dapat dioptimalkan menjadi *move* atau dieliminasi sepenuhnya (*copy elision*).
- cctor untuk kelas `X` dideklarasikan sebagai `X(const X&);`

```
class Stack {
  public:
    // ... ctor
    Stack (const Stack&);   // cctor
    // ... dtor, dst.
};
```

```
#include <algorithm>

Stack::Stack(const Stack& other)
    : capacity_{other.capacity_},
      top_{other.top_},
      data_{new int[other.capacity_]}
{
    std::copy(other.data_, other.data_ + other.top_, data_);
}
```

Copy Assignment (“operator=”)

- **Dua Situasi Berbeda**

```
Stack s1(10);
Stack s2 = s1;    // copy constructor (objek baru)
```

vs.

```
Stack s1(10);
Stack s2(5);
s2 = s1;          // copy assignment (objek sudah ada)
```

operator= vs. cctor

- **Copy constructor**
  - Membangun identitas baru
  - Mengalokasikan resource baru
  - Menyalin state
- **Copy assignment**
  - Objek sudah ada
  - Harus melepaskan resource lama
  - Lalu menyalin resource baru

Contoh Implementasi

```
Stack& Stack::operator=(const Stack& other) {
    if (this == &other) return *this;   // self-assignment guard

    int* new_data = new int[other.capacity_];
    std::copy(other.data_, other.data_ + other.top_, new_data);

    delete[] data_;     // lepaskan resource lama

    data_ = new_data;
    capacity_ = other.capacity_;
    top_ = other.top_;

    return *this;
}
```

- Tidak Sederhana, karena:
  - Harus menangani self-assignment.
  - Harus aman jika alokasi gagal.
  - Harus menjaga invariant tetap valid.

Move Semantics (sejak C++11)

- Motivasi: *copy* mahal, padahal sering dilakukan saat return dari fungsi / *method*:

```
Stack make_stack() {
    Stack _s;   // diciptakan
    _s.push(1);
    _s.push(2);
    return _s;   // old C++: _s di-copy ke s (operasi yg mahal)
} // _s dihancurkan

Stack s = make_stack();
```

- *Move semantics* menambahkan *move* untuk ctor dan operator=

  `Stack(Stack&&)` dan `Stack& operator=(Stack&&)`

```
#include "stack.hpp"
void f1(const Stack& s);  // pass by reference: tidak membuat objek baru
void f2(Stack s);         // pass by value: membuat objek baru
(copy/move)
Stack make_stack() {
    Stack s(10);
    return s;  // move atau copy elision (C++17+)
}

main () {
  Stack s1{20};

  f1(s1);   // tidak ada objek baru
  f2(s1);   // copy constructor (s1 adalah lvalue)

  Stack s2{20};
  Stack s3 = s2;             // copy constructor
  Stack s4 = std::move(s2);  // move constructor -- s2 tetap valid tapi
kosong
  Stack s5 = make_stack();   // move / copy elision
}
```

Const & Static Member

Const Member

- Keyword `const` dapat diterapkan pada atribut maupun method.
- Pada atribut: nilai atribut tersebut akan tetap sepanjang waktu hidup objeknya.
  - Pengisian nilai awal harus dilakukan pada saat objek tersebut diciptakan, yaitu melalui *constructor initialization list*.
- Pada method: method tersebut tidak bisa mengubah (status) data member yang dimiliki oleh kelasnya.
- Object juga dapat ditandai sebagai `const`
  - hanya boleh memanggil const method, untuk memastikan bahwa status object tidak berubah.

Anggota Statik

- Anggota statik adalah anggota yang “dimiliki” oleh kelas, bukan oleh objek dari kelas tersebut.
- Anggota statik juga dapat berupa atribut maupun method.

Atribut Statik

- Setiap objek dari kelas memiliki sendiri salinan atribut non-statik.
- Atribut statik **dipakai bersama** oleh seluruh objek dari kelas tersebut.
  - Tidak bergantung pada keberadaan objek.
- Di old C++, inisialisasi dilakukan di implementation file:
  `Stack::n_stack = 0;`
  - Di modern C++, inline di header:

```
class Stack {
  public:
    // ... method lain
  private:
    inline static int n_stack = 0;
    // ... atribut & method lain
};
```

Method Statik

- Method yang hanya mengakses anggota statik dapat dideklarasikan static di dalam file: stack.hpp

```
class Stack {
  // ...
  public:
    static int numStackObj();
  private:
    static int n_stack;
};
```

- Di dalam file: stack.cpp

```
int Stack::numStackObj() {
  // kode mengakses hanya
  // atribut statik
  return n_stack;
    // jika mau explicit
scope:
    //   Stack::n_stack
    // BUKAN
    //   this->n_stack
}
```

Sifat Anggota Statik

- method statik dapat dipanggil tanpa melalui objek dari kelas tersebut, misalnya:

```
if (Stack::numStackObj() > 0) {
  printf("...");
}
```

- method statik tidak memiliki pointer implisit `this`
- Atribut statik diinisialisasi tanpa perlu adanya objek dari kelas tersebut.

Friend & Nested Class

Friend (1)

- Friend = pemberian hak pada fungsi/kelas untuk mengakses anggota *non-public* suatu kelas

```
class B { // kelas "pemberi izin"
  friend class A;
  friend void  f(int, char*);
}
```

  - *Friend* bersifat satu arah
  - Seluruh member kelas `A`, dan fungsi `f` dapat mengakses anggota *non-public* dari kelas `B`

Friend (2)

- Friend adalah mekanisme *controlled breach of encapsulation*, bukan fitur utama OOP.
  - Gunakan untuk operator overloading.
  - Gunakan bila ada tightly coupled abstraction pair.
- Fungsi friend merupakan fungsi di luar kelas sehingga objek parameter aktual mungkin dilewatkan secara *call-by-value*
  - Akibatnya operasi yang dilakukan terhadap objek bukanlah objek semula

Nested Class (1)

- Nested class adalah:
  - Kelas yang didefinisikan di dalam scope kelas lain.
  - Detail representasi internal.
  - Gunakan untuk menyembunyikan tipe internal seperti `Node` atau `Iterator`.
  - Ini konsisten dengan prinsip information hiding.

```
class List;

class ListElem { // tidak
nested
  friend class List;
  public:
    // ...
  private:
    // ...
};

class List {
  public:
    // ...
  private:
    // ...
};
```

Nested Class (2)

- Pemakai kelas `List` **tidak perlu** mengetahui keberadaan kelas `ListElem`
  - Yang perlu diketahui: adanya layanan untuk menyimpan nilai (integer) ke dalam list maupun untuk mengambil nilai dari list
- Kelas `ListElem` dapat dijadikan sebagai *nested class* di dalam kelas `List`

```
class List {
  public:
    // ...
  private:
    // ...
    class ListElem { //
nested
      public:
        // ...
      private:
        // ...
    };
};
```

Method pada Nested Class

- Contoh `List` dan `ListElem`
  - Nama scope yang digunakan untuk kelas `ListElem` adalah `List::ListElem::`, bukan hanya `ListElem::`
    - Jika merupakan kelas generic dengan parameter generic `Type`, nama scope menjadi `List<Type>::ListElem`

Penutup

Rangkuman

- Dalam OO Analysis:
  - Kita memodelkan objek dan tanggung jawab.
- Dalam OO Design:
  - Kita mendesain struktur kelas dan relasi.
- Dalam OO Programming:
  - Kita merealisasikan lifecycle, ownership, dan interaksi secara konkret.
- Modern C++ membantu kita:
  - Mengelola lifecycle otomatis.
  - Menghindari raw resource management.
  - Fokus pada desain, bukan pada pointer.

Takeaway

- Jika Anda merasa harus menulis dtor, cctor, dan move constructor, mungkin desain Anda terlalu dekat dengan *raw resource*.
  - Biasanya dilakukan saat kontrol resource yang lebih eksplisit dibutuhkan:
    - Mengelola resource mentah (raw pointer, file descriptor, handle OS)
    - Menerapkan kebijakan ownership khusus (unique ownership, shared ownership, move-only type)
    - Mengoptimalkan struktur data internal yang tidak dapat diwakili oleh tipe standar
- Modern C++ mendorong kita untuk mendesain kelas yang mengikuti *Rule of Zero*.